Week 3 - Day 4:

Implement Full CRUD Operat

Overview

In this day, I implemented the full CRUD operations for the main resource (**Books**) in the ASP.NET Core Web API project. I created endpoints for creating, reading, updating, and deleting books, and tested them using Postman with both successful and error scenarios.

1. Create Book Endpoint (POST)

Implemented a Create endpoint to add a new book.

Endpoint:

```
POST /api/v1/day4/books
```

The endpoint:

- Receives book data from the request body
- Saves the new book using Entity Framework Core
- Returns the created resource

Response:

```
201 Created
```

The response includes:

- Created book data
- Location header containing the URL of the new resource

Example:

```
Location: /api/v1/day4/books/{id}
```

---

2. Get All Books Endpoint (GET)

Implemented an endpoint to retrieve all books.

Endpoint:

```
GET /api/v1/day4/books
```

The endpoint returns a list of all books stored in the database.

Response:

```
200 OK
```

---

3. Get Book By Id Endpoint (GET)

Implemented an endpoint to retrieve a specific book using its ID.

Endpoint:

```
GET /api/v1/day4/books/{id}
```

The endpoint:

- Searches for the book by ID
- Returns the book if it exists
- Returns an error if the book does not exist

Successful response:

```
200 OK
```

Error response:

```
404 Not Found
```

Example error:

```
{
    "message":"Book not found"
}
```

---

4. Update Book Endpoint (PUT)

Implemented an update endpoint to modify existing book information.

Endpoint:

```
PUT /api/v1/day4/books/{id}
```

The endpoint handles:

Successful update

Response:

```
200 OK
```

Invalid input

Example:

- Negative price value

Response:

```
400 Bad Request
```

Book not found

Response:

```
404 Not Found
```

---

5. Delete Book Endpoint (DELETE)

Implemented a delete endpoint to remove a book.

Endpoint:

```
DELETE /api/v1/day4/books/{id}
```

Successful deletion:

```
204 No Content
```

If the book does not exist:

```
404 Not Found
```

---

6. API Testing Using Postman

Tested all CRUD endpoints using Postman.

Test cases included:

Successful Requests

- Get all books
- Get book by ID
- Create book
- Update book
- Delete book

Error Cases

Added deliberate error scenarios:

- Get book with invalid ID → 404
- Create book with missing required fields → 400
- Update book with invalid price → 400
- Update non-existing book → 404
- Delete non-existing book → 404

Tools Used

- ASP.NET Core Web API
- Entity Framework Core
- C#
- Postman